

MÓDULO 03

JAVASCRIPT

Lógica do Formulário de Cadastro

Captura de dados, manipulação de objetos e localStorage

Introdução

O Foco da Aula?

Este é o script mais enxuto do nosso projeto, mas ele é poderoso. Vamos introduzir conceitos novos e fundamentais para o desenvolvimento web moderno.

Tópicos:

- Captura inteligente de dados de formulário.
- O uso do operador spread (...).
- Geração de IDs automáticos.

Atualizando o Carrinho (IIFE)

JS

Bloco de código:

```
const atualizarValorCarrinho = () => {  
  document.querySelector(".cart-count").textContent = (  
    JSON.parse(localStorage.getItem("carrinho")) || []  
  ).length;  
})();
```

Explicação: O padrão **IIFE** (Immediately Invoked Function Expression) significa que a função é definida e executada no mesmo instante. Usamos isso para atualizar o ícone do carrinho com os itens salvos logo que a página carrega. O `|| []` evita erros caso o carrinho esteja vazio.

Interceptando o Envio

J S

Bloco de código:

```
const salvarProduto = (event) => {  
  event.preventDefault();  
  // lógica de salvamento...  
}
```

Explicação: O `event.preventDefault()` é a linha mais importante de um formulário com JavaScript. Ele impede o comportamento padrão do navegador (que seria recarregar a página). Sem ele, a página piscaria e perderíamos os dados antes mesmo de salvá-los!

Lendo Dados Existentes

J S

Bloco de código:

```
const produtos = JSON.parse(localStorage.getItem("produtos") || "{}");
```

Explicação: Antes de salvar um novo produto, precisamos resgatar os que já existem. O uso do `|| "{}"` é uma trava de segurança crucial: se não houver nada salvo no `localStorage`, o **JSON.parse** recebe um objeto vazio em vez de um valor nulo, evitando a quebra da aplicação.

O Poder do FormData

JS

Bloco de código:

```
const dados = Object.fromEntries(new FormData(event.target));
```

Explicação: Em vez de capturar cada campo manualmente, fazemos tudo de uma vez. O *event.target* referencia o formulário. O **FormData** lê o atributo *name* de todos os inputs, e o **Object.fromEntries()** converte tudo em um objeto JavaScript limpo e pronto para uso.

Gerando o ID do Produto

JS

Bloco de código:

```
const getId = () => {  
  const id = parseInt(localStorage.nextId || "0");  
  localStorage.setItem("nextId", `${id + 1}`);  
  return id;  
};
```

Explicação: Para gerar um ID único e crescente, lemos o último número usado no *localStorage*. O uso do *parseInt()* é obrigatório aqui, pois o *localStorage* sempre guarda informações em formato de texto (string) e precisamos fazer cálculos matemáticos (*id + 1*).

O Operador Spread

J S

Bloco de código:

```
produtos[id] = { id: id, ...dados };
```

Explicação: O operador spread (...) copia automaticamente todas as propriedades que capturamos do formulário (nome, preço, foto, etc.) e as "espalha" para dentro do novo objeto do produto. É muito mais prático do que digitar campo por campo!

Salvando e Limpando

J S

Bloco de código:

```
localStorage.setItem("produtos", JSON.stringify(produtos));  
event.target.reset();
```

Explicação: Convertendo nosso objeto atualizado para texto com **JSON.stringify**, salvamos no banco do navegador. Logo em seguida, chamamos **.reset()**, um método nativo que limpa todos os campos da tela de uma só vez.

Rolagem Suave

J S

Bloco de código:

```
window.scrollTo({ top: 0, behavior: "smooth" });  
notificar();
```

Explicação: Para garantir uma boa experiência de navegação, rolamos a página de volta ao topo de forma animada (***behavior: "smooth"***) e chamamos a função que exibirá o aviso de sucesso para o usuário.

A Função Notificar (Toast)

J S

Bloco de código:

```
const notificar = () => {  
  const toast = document.querySelector(".toast");  
  toast.classList.add("mostrar");  
  setTimeout(() => {  
    toast.classList.remove("mostrar");  
  }, 3000);  
};
```

Explicação: Adicionamos a classe **mostrar** para fazer o aviso aparecer na tela. Usamos o **setTimeout** para aguardar 3000 milissegundos (3 segundos) e remover a classe automaticamente, fazendo a notificação sumir sem intervenção do usuário.

Resumo E Desafio

Conclusão e Prática

Entendemos funções IIFE, *preventDefault* no envio, captura de dados com *FormData*, operador spread, conversão numérica com *parseInt* e o método *.reset()*.

Missão do Aluno:

1. Impedir que um produto seja salvo se o preço for negativo ou zerado.
2. Adicionar um *console.log* antes de salvar para inspecionar os dados capturados.
3. Exibir um pequeno resumo visual do produto na tela logo após salvar.
4. Adicionar um pop-up de confirmação antes de limpar o formulário.